“国芯国模”黑客松 · 技术报告

赛题 T04 语音模型适配 ｜ 卡型 K100-AI（K500SM_AI 64GB 单卡） ｜ vLLM 0.21.0 ｜ bf16

1. 简介

本报告记录 **Qwen3-ASR-1.7B**、**Qwen3-ASR-0.6B**、**Qwen3-ForcedAligner-0.6B** 三个语音模型在 海光 K100-AI（实测实例 K500SM_AI 64GB 单卡）上的适配与优化结果，覆盖语音识别（转写）与强制对齐（词级时间戳） 两类任务。

核心结论：三个模型均在该卡上跑通并给出可复现的实测数据，**未修改任何模型代码、未升级任何软件包**， 全部复用镜像自带的 HCU 版 torch / triton / vllm-hcu 与 DTK；适配工作集中在运行时装配与三个启动参数的确定上。

关键实测：单并发下 Qwen3-ASR-1.7B 达 21.97 倍实时、32 并发达 208 倍实时；0.6B 分别为 37 倍与 225 倍实时； 强制对齐模型对 7.18 s 英文音频输出 16 个词级时间戳、对 5.62 s 中文音频输出 13 个逐字时间戳，均严格单调且落在音频时长内。

> 提交仓库：**https://github.com/devinhuang2001/hygon-hackathon-2026**
> 仓库内 `submissions/01`、`02`、`08` 三个目录即为本报告对应的完整提交包（含脚本与原始证据）。

2. 验证环境

镜像	`harbor.sourcefind.cn:5443/hcu/admin/base/vllm:0.21.0-ubuntu22.04-dtk2604-py3.10`
卡型	K500SM_AI 64 GB × 1（120 SM），按赛事口径计入 K100-AI
节点	SCNet 超算互联网 Notebook，Hygon C86 7185 32C / 503 GB RAM
Python / torch	3.10.21 / `2.10.0+das.opt1.dtk2604.2608141812`（HIP 6.3.26113）
triton / vllm	`3.6.0+das185.dtk2604` / `0.21.0+das.dtk2604`（vllm-hcu `0.21.0+das`）
transformers	5.5.0
精度 / 并行	bf16 / 单卡（TP=1, PP=1）
服务端口	8000（OpenAI 兼容接口）

说明：该实例无容器运行时（无 docker / podman），镜像层经 harbor 匿名拉取后在实例内解包为 rootfs（24 层 / 7.87 GB → 解包 31 GB），运行时以宿主内核 + 镜像内 DTK/Python 包执行，驱动 `/opt/hyhal` 由平台注入—— 与容器内的实际形态一致（DTK 与包来自镜像，驱动来自宿主）。

3. 适配优化方案

3.1 基础镜像与软件包

使用 `docker_images.md` 中 vLLM 0.21 的推荐镜像；**未升级任何软件包**，`requirements-extra.txt` 为空。 镜像自带 torch / triton / vllm / vllm-hcu / flash_attn / aiter / lmslim 全套海光版本。

3.2 三个必须处理的运行时装配

现象	根因	处理
`ModuleNotFoundError: vllm...fused_moe.modular_kernel`	镜像内 `vllm_hcu` 用**绝对**符号链接覆盖 vllm 的部分模块，解包后指向 `/usr/local/...` 失效	把指向镜像绝对路径的链接改指向 `$ROOTFS` 同一路径（`run.sh` 内幂等执行）
`get hyhal driver version error` / hcusmi 初始化失败	镜像不含 `/opt/hyhal`（驱动用户态由平台注入，等同 nvidia-container-toolkit）	补入宿主 `/opt/hyhal`，并放进 `LD_LIBRARY_PATH`
`HSACOError: File operation failed: ... .amdgcn`，表现为 EngineCore 启动失败	triton 的 HCU 后端从 `ROCM_PATH` 下找 `aillvm`；宿主 DTK 无该目录，回退到 `llvm` 导致编译产物缺失	`ROCM_PATH` 指向**镜像内** DTK（含 `aillvm`），驱动仍用宿主 `/opt/hyhal`

3.3 强制对齐模型的三项专属设置

Qwen3-ForcedAligner-0.6B 的 `config.json` 中标称 `architectures=Qwen3ASRForConditionalGeneration`，但实际是 token classification 头。三项设置缺一不可，否则分别报「按生成模型加载失败」「走生成路径」「max_model_len 校验失败」：

```
--runner pooling
--hf-overrides '{"architectures":["Qwen3ASRForcedAlignerForTokenClassification"]}'
--max-model-len 8192        # 模型 max_position_embeddings 上限，写 32768 直接报错
```

官方用 `qwen_asr` 包的 `Qwen3ForcedAligner.align()`；本环境无该包（PyPI 镜像亦无），故按官方源码 `qwen_asr/inference/qwen3_forced_aligner.py` 逐段复刻算法到 vLLM pooling 路径：

- 输入构造：`<|audio_start|><|audio_pad|><|audio_end|>` + `"<timestamp><timestamp>".join(words) + "<timestamp><timestamp>"`
- 取 `timestamp_token_id=151705` 位置的分类 argmax，`时间(ms) = 索引 × timestamp_segment_time`（本模型 80 ms/格）
- 用最长递增子序列平滑异常跳变后，按 (start, end) 成对切给每个词；中文按官方 CJK 规则逐字切分

3.4 升级步骤

```
解包镜像 → bash run.sh（装配环境 + 起服务 + 轮询 /health）→ bash env_check.sh → bash smoke.sh → bash bench.sh
```

4. 服务启动脚本

4.1 语音识别（两个 ASR 模型，仅模型路径不同）

```
python3 -m vllm.entrypoints.openai.api_server \
  --model /root/hygon/models/asr_orig \
  --served-model-name qwen3-asr-1.7b \
  --trust-remote-code --dtype bfloat16 \
  --max-model-len 32768 --gpu-memory-utilization 0.90 \
  --host 127.0.0.1 --port 8000
```

参数理由：`--dtype bfloat16` 与官方权重一致、匹配镜像内 flash-attn 路径；`--max-model-len 32768` 为模型原生上下文（单条音频远小于此，KV cache 按实际长度分配）；`--gpu-memory-utilization 0.90` 单卡独占， 留 10% 给 DTK 运行时与编译缓存，实测稳态显存 60.2 GB / 64 GB；未开 `--enforce-eager`，走默认 torch.compile + CUDA Graph（首次启动一次性编译，缓存可复用）。

4.2 强制对齐

```
python3 -m vllm.entrypoints.openai.api_server \
  --model /root/hygon/models/fa06 \
  --served-model-name qwen3-forcedaligner-0.6b \
  --runner pooling \
  --hf-overrides '{"architectures":["Qwen3ASRForcedAlignerForTokenClassification"]}' \
  --trust-remote-code --dtype bfloat16 \
  --max-model-len 8192 --gpu-memory-utilization 0.35 \
  --host 127.0.0.1 --port 8000
```

`--gpu-memory-utilization 0.35` 足够（0.6B 分类头 + 长音频 KV），留余量便于与其它模型共用一张卡。

5. Smoke 测试

5.1 语音识别

```
curl http://localhost:8000/v1/audio/transcriptions \
  -F "file=@data/zh.mp3" -F "model=qwen3-asr-1.7b"

# 中文 → {"text":"開放時間：早上九點至下午五點。","usage":{"type":"duration","seconds":6}}
# 英文 → {"text":"The tribal chief then called for the boy, and presented him
#          with fifty pieces of gold.","usage":{"type":"duration","seconds":8}}
```

中英两条均为逐字正确的原文转写（中文样例本身是繁体中文字面，模型输出亦为繁体，未改写）。 0.6B 版本输出一致，仅英文少一个逗号，属规格差异。

5.2 强制对齐

```
python fa_align.py data/en.mp3 "The tribal chief then called for the boy and \
presented him with fifty pieces of gold." English

{"text": "The",    "start": 1.12, "end": 1.20}
{"text": "tribal", "start": 1.20, "end": 1.76}
{"text": "chief",  "start": 1.76, "end": 2.08}
...
{"text": "fifty",  "start": 4.96, "end": 5.20}
{"text": "gold.",  "start": 6.96, "end": 7.04}        # 音频 7.18 s
```

中文（5.62 s 音频，逐字）：`開 0.72-0.96 / 放 0.96-1.12 / … / 點 4.80-5.12`。 两条都满足：时间戳严格单调、首尾落在音频时长内、词/字数与文本一致。

6. 精度测评

> **本轮未提交精度分数。**赛事要求 `verify.py` 按官方 `baselines/<模型>.json` 写死的子集题数、max_tokens、思考开关与官方 GPU 基线分现场跑分；该基线文件与评测子集随分题材料下发，我方未获取到。

已尝试的获取途径：飞书知识空间返回 `131006 permission_denied`（节点所属空间未授权）； 文档搜索「分题材料 / baselines / gsm8k.jsonl」0 命中；实例与共享存储全盘检索无 `baselines/`、无评测子集。

按赛事红线（伪造日志取消资格），我们选择留空并写明补齐方式，而不是给出不可复现的数字。`verify.py` 已实现官方判分逻辑（容差 2%、只罚劣化不罚提升、`--selfcheck` 自检通过）， 拿到官方 `baselines/` 与 `data/` 后执行 `source env.sh && python verify.py` 即可产出 `verify_result.json`。强制对齐的 AAS(ms) 同样需要官方参考输出才能计算。

7. 性能参考

7.1 语音识别（自定口径，官方 T04 测法随分题材料下发）

指标：RTF = 墙钟时间 / 音频时长（越小越好）、吞吐 = audio-s/s。测试集为 3 条公开样例（6 s 中文、8 s 英文、 6 s 其他），按并发重复提交，全部 HTTP 200。

模型	并发	请求数	墙钟	音频总长	RTF	吞吐	QPS
Qwen3-ASR-1.7B	1	63	19.12 s	420 s	0.046×	21.97	3.30
	8	48	2.68 s	320 s	0.008×	119.53	17.93
	32	96	3.07 s	640 s	0.005×	208.23	31.23
Qwen3-ASR-0.6B	1	63	11.34 s	420 s	0.027×	37.04	5.56
	8	48	2.51 s	320 s	0.008×	127.26	19.09
	32	96	2.84 s	640 s	0.004×	225.05	33.76

0.6B 在单并发下比 1.7B 快 1.7 倍，但 32 并发下差距收敛到 8%——说明并发上到 32 后瓶颈不在模型算力侧， 而在服务链路（HTTP/调度），继续加并发收益有限。

7.2 强制对齐

引擎初始化	8.74 s（profile + KV cache + warmup，`enforce_eager`）
单条 7.18 s 音频 / 16 词，进程冷启动到出结果	120 s（绝大部分是 Python / torch / vllm 导入）
稳态推理延迟	本轮未做重复压测，未测

需要说明：120 s 是**进程级冷启动**，不是稳态延迟；稳态延迟应在服务常驻后用多次请求取中位数，本轮未做。

8. 注意事项

- **`-hf` 后缀的 ASR 权重不能用**：`Qwen3-ASR-1.7B-hf` 的 707 个张量全部带 `model.` 前缀， vLLM 0.21 加载报 `ValueError: There is no module or parameter named 'model'`。必须用非 hf 版（`thinker.` 前缀）。这一点会浪费大量排查时间，建议写进官方文档。
- **三个装配缺一不可**：符号链接重指向、补 `/opt/hyhal`、`ROCM_PATH` 指向镜像内 DTK。 第三个最隐蔽——现象是 EngineCore 启动失败，容易误判为「模型不支持」。
- **纯文本请求不能当文本模型用**：ASR 模型收到无音频的文本请求会返回 `<asr_text>` 类空结果， 接口可用但语义无意义。
- **音频必须解码成数组传入**：vLLM 的 `multi_modal_data={"audio": path}` 传字符串会抛 `AssertionError ... got: '/'`；且镜像内 `torchaudio.load` 因缺 `torchcodec` 不可用， 用 `ffmpeg -f f32le -ac 1 -ar 16000` 管道解码最稳。
- **时间戳掩码要按后缀对齐**：音频占位符会在引擎侧展开（本例预测 148 个位置、纯文本 52 个）， 按纯文本 token 数取位置会取错；早期写法能出数但同一句音频起点会从 1.12 s 漂到 2.96 s。
- **首次启动耗时不是卡死**：ASR 约 2–5 分钟（torch.compile）；强制对齐在 `enforce_eager` 下仅 8.7 s。 建议用「轮询 /health + 进程存活检查」代替固定 sleep。
- aiter 会提示 `NUMA balancing is enabled, which may cause errors`，本实例无权限写 `/proc/sys/kernel/numa_balancing`，实测未影响结果，保留原样。

本报告所有数字均来自仓库 `evidence/` 目录下的原始日志与压测 JSON，可逐条复现。